

Machine Learning Lab

Yukun Jiao

2025-08-27

Lab 2

Part 1:

1.

```
library(data.table)
library(ggplot2)
library(caret)
library(glmnet)
library(randomForest) # New (random forest)
library(rpart.plot)    # New (to plot decision trees)
library(gbm)           # New (boosting)
```

```
sna_data <- fread(file = "~/Documents/ML/Labs/Lab2/Kaggle_Social_Network_Ads.csv")
sna_data$Purchased <- factor(sna_data$Purchased, levels = c(0,1))
sna_data <- sna_data[, -"user_id"]
```

2. k-NN

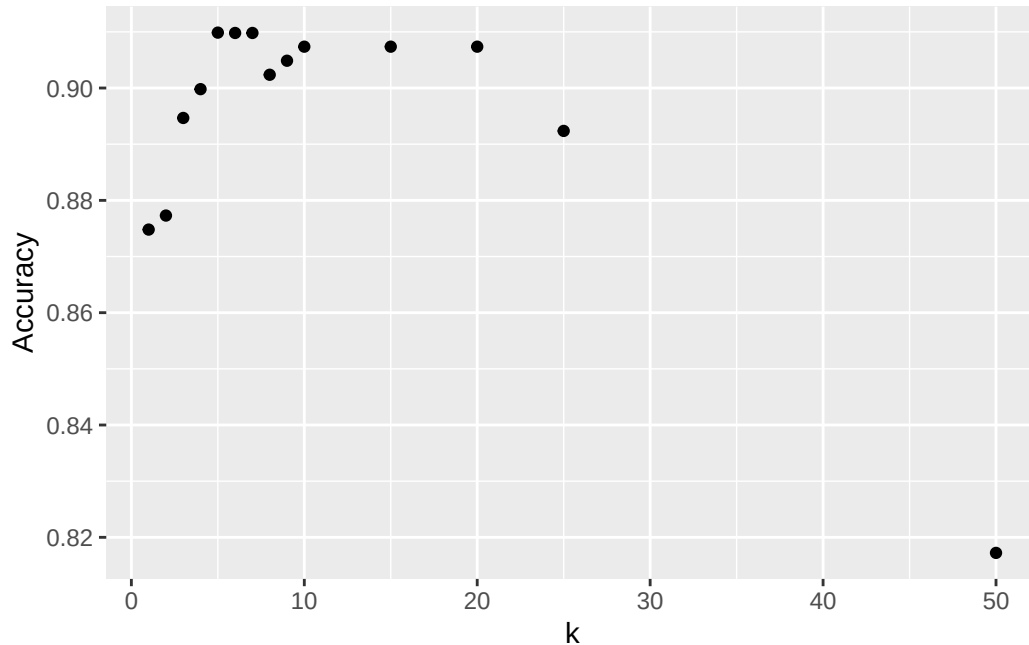
```
set.seed(1984)
tc <- caret::trainControl(method = 'cv', number = 10)
k_value <- c(1,2,3,4,5,6,7,8,9,10,15,20,25,50)
knn_model <- caret::train(Purchased ~ .,
                           data = sna_data,
                           tuneGrid=expand.grid(k=k_value),
```

```
method = "knn",
trControl = tc,
preProcess = c("center", "scale"))
```

```
knn_model$results
```

	k	Accuracy	Kappa	AccuracySD	KappaSD
1	1	0.8747983	0.7259415	0.02716813	0.06237010
2	2	0.8772952	0.7332317	0.05128252	0.11047128
3	3	0.8946732	0.7695092	0.04476222	0.10053370
4	4	0.8998014	0.7809367	0.04748517	0.10744353
5	5	0.9098655	0.8056182	0.03598877	0.07694083
6	6	0.9098014	0.8056594	0.04482470	0.09674281
7	7	0.9098014	0.8059659	0.04482470	0.09606171
8	8	0.9023624	0.7899090	0.04025547	0.08857650
9	9	0.9048624	0.7948695	0.04567872	0.09982811
10	10	0.9073624	0.7997813	0.04606552	0.10097222
11	15	0.9073624	0.7991482	0.04754914	0.10414136
12	20	0.9073624	0.7973870	0.04453249	0.09929468
13	25	0.8923624	0.7595190	0.05921431	0.14698016
14	50	0.8172311	0.5594800	0.05838184	0.15088819

```
ggplot(knn_model$results, aes(x = k, y = Accuracy)) + geom_point()
```



As k increases, the test accuracy first rises and then falls, reaching its highest value at $k = 5$ and its lowest at $k = 50$.

Compared with GAM, the accuracy is almost the same, since both GAM and KNN can capture nonlinear relationships. In contrast, their performance is significantly better than standard linear regression, which fails to capture nonlinear relationships.

I prefer GAM, because it can also capture nonlinear relationships while allowing visualization of them, and it provides interpretability regarding the contribution of each feature.

3.

```
snaa_data <- fread("~/Documents/ML/Labs/Lab2/Kaggle_Social_Network_Ads_Augmented.csv")
snaa_data$Purchased <- factor(snaa_data$Purchased, levels = c(0,1))
snaa_data <- snaa_data[, -"user_id"]
```

4.

```

set.seed(1984)
tc <- caret::trainControl(method = 'cv', number = 10)
knn_model_1 <- caret::train(Purchased ~ .,
                             data = snaa_data,
                             tuneGrid=expand.grid(k=k_value),
                             method = "knn",
                             trControl = tc,
                             preProcess = c("center", "scale"))

knn_model_1$results

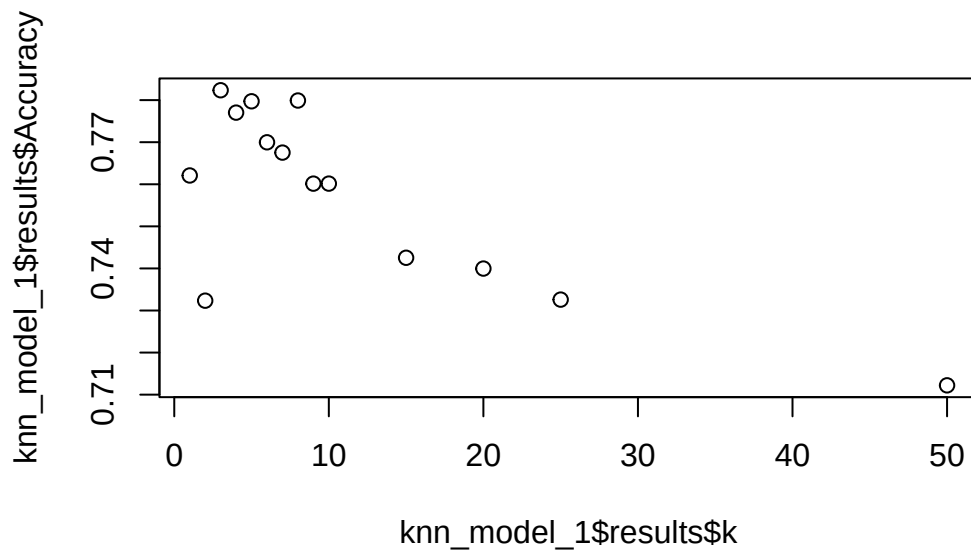
```

	k	Accuracy	Kappa	AccuracySD	KappaSD
1	1	0.7620872	0.4643836	0.07319289	0.1622648
2	2	0.7323374	0.4074837	0.06216858	0.1449631
3	3	0.7823468	0.4955507	0.06516181	0.1541910
4	4	0.7770294	0.4785734	0.08052121	0.1900509
5	5	0.7797186	0.4800246	0.04083580	0.1058651
6	6	0.7699719	0.4572698	0.03881663	0.1003090
7	7	0.7675297	0.4456461	0.04540290	0.1157449
8	8	0.7799078	0.4770678	0.03930064	0.1089182
9	9	0.7601517	0.4167426	0.06044680	0.1564926
10	10	0.7601548	0.4170429	0.05326606	0.1425435
11	15	0.7425328	0.3601497	0.05120701	0.1552535
12	20	0.7399687	0.3470468	0.03954183	0.1245335
13	25	0.7325876	0.3270424	0.04929409	0.1462534
14	50	0.7122123	0.2510304	0.04945088	0.1442128

```

plot(knn_model_1$results$k, knn_model_1$results$Accuracy)

```



After the same steps, the accuracy decreased due to the inclusion of irrelevant variables. This is because the newly added variables contain too much noise, and KNN lacks the ability to perform feature selection, making it vulnerable to the influence of noise.

5.

Yes, because the KNN method is sensitive to noise variables. In contrast, decision trees perform better as they select features through recursive binary splitting to minimize RSS, and they are also well suited to capturing nonlinear relationships.

6.

6a

Because a very large value of minbucket may lead to underfitting, while a very small value of minbucket may lead to overfitting.

6b

```

set.seed(1984)
tc <- caret::trainControl(method = 'cv', number = 10)
rpart_model <- caret::train(Purchased ~ .,
                             data = snaa_data,
                             tuneGrid=expand.grid(cp=c(0,0.01,0.025,0.05,0.10,0.25,0.5,1.0)),
                             control = rpart.control(minbucket=10),
                             method = "rpart",
                             trControl = tc)

rpart_model$results

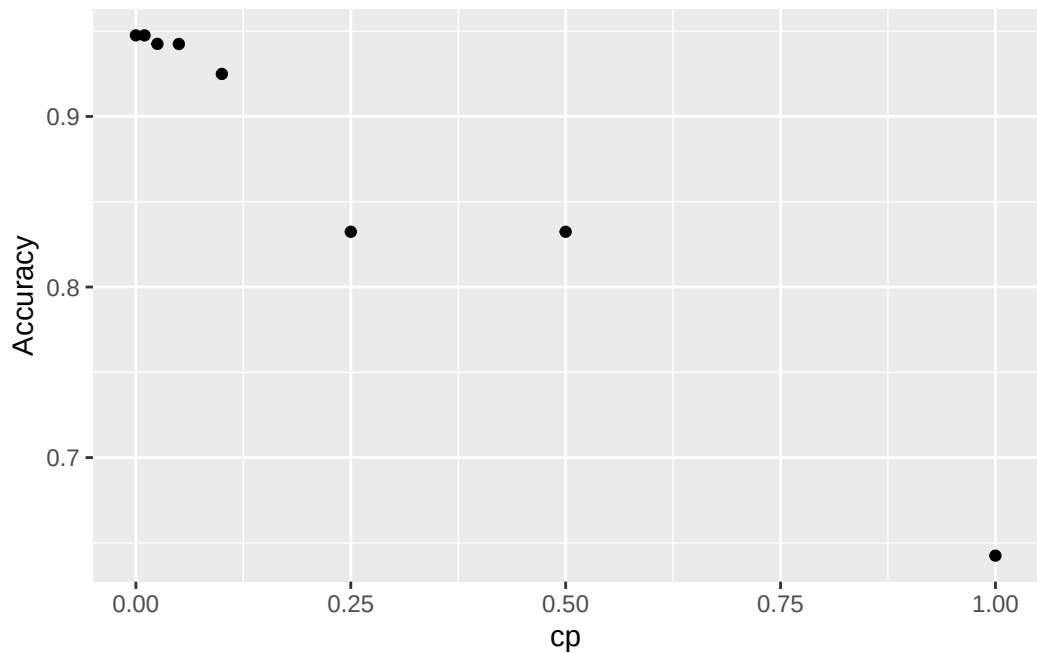
```

	cp	Accuracy	Kappa	AccuracySD	KappaSD
1	0.000	0.9475578	0.8820295	0.03618656	0.08287603
2	0.010	0.9475578	0.8820295	0.03618656	0.08287603
3	0.025	0.9425578	0.8727634	0.03541969	0.07899275
4	0.050	0.9424969	0.8748758	0.03735899	0.08092178
5	0.100	0.9249265	0.8387310	0.03557164	0.07559537
6	0.250	0.8324109	0.6106415	0.06808156	0.17450342
7	0.500	0.8324109	0.6106415	0.06808156	0.17450342
8	1.000	0.6425172	0.0000000	0.01065601	0.00000000

```

ggplot(rpart_model$results, aes(x=cp, y=Accuracy)) + geom_point()

```



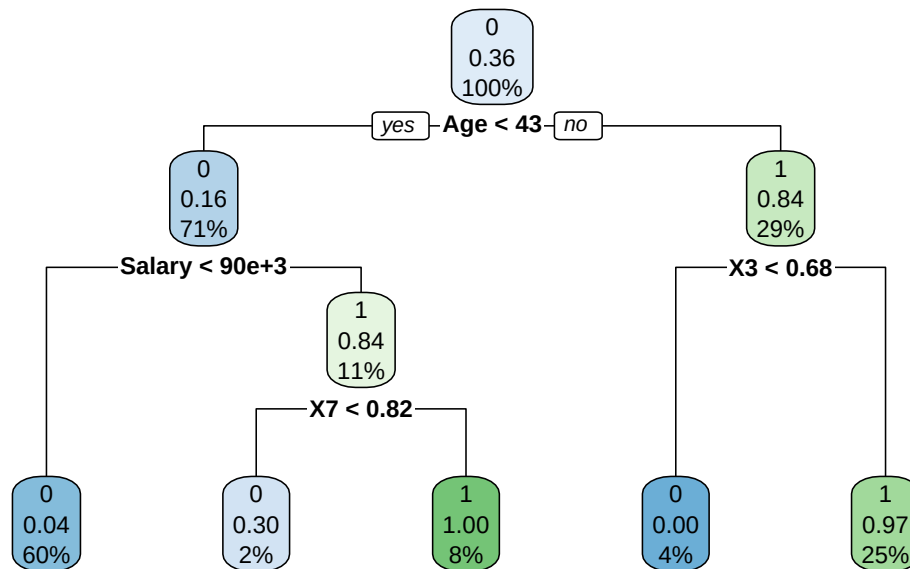
CP corresponds to alpha, which is the penalty applied to the number of leaf nodes in a tree. Generally, the test accuracy first increases and then decreases as CP changes. A smaller CP results in a more complex tree, increasing the risk of overfitting. A larger CP produces a simpler tree, increasing the risk of underfitting.

According to the model results, the best CP values are 0 and 0.010, with the model achieving an accuracy of 0.9474875.

Comparing the accuracy at the best CP with that of KNN, we can see that the decision tree's highest test accuracy is significantly higher than KNN's best test accuracy. I attribute this difference to KNN being more sensitive to noisy variables.

6c

```
best_dt_model <- rpart_model$finalModel
rpart.plot(best_dt_model)
```



There are 4 internal nodes and 5 terminal nodes.

Each path from the root to a leaf represents a rule.

These rules are as follows:

$$R_1 = \{X \mid \text{Age} < 43, \text{Salary} < 90,000\}, \quad \hat{Y} = 0$$

$$R_2 = \{X \mid \text{Age} < 43, \text{Salary} \geq 90,000, x_7 < 0.82\}, \quad \hat{Y} = 0$$

$$R_3 = \{X \mid \text{Age} < 43, \text{Salary} \geq 90,000, x_7 \geq 0.82\}, \quad \hat{Y} = 1$$

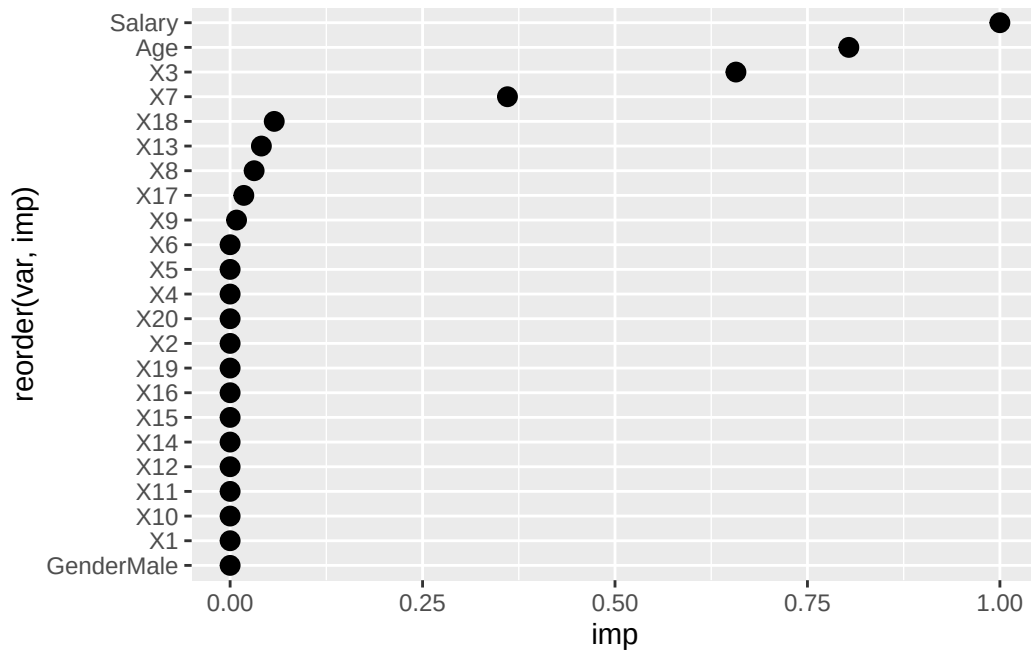
$$R_4 = \{X \mid \text{Age} \geq 43, x_3 < 0.68\}, \quad \hat{Y} = 0$$

$$R_5 = \{X \mid \text{Age} \geq 43, x_3 \geq 0.68\}, \quad \hat{Y} = 1$$

The new variables x_3 and x_7 appear in the tree.

6d

```
varimp_object <- caret::varImp(best_dt_model)
varimp_dt <- data.table(var=rownames(varimp_object),
                        imp=varimp_object$Overall)
varimp_dt <- varimp_dt[order(imp,decreasing = T)]
varimp_dt[,imp := imp / max(imp)] # Making variable importance into a relative measure
ggplot(varimp_dt,aes(y=reorder(var, imp),x=imp)) + geom_point(size=3)
```



The larger the variable importance value, the more important the variable; the smaller the value, the less important it is, or it may even be noise. From the figure, it can be observed that

Salary, Age, x_3 , and x_7 have relatively high importance, while the importance of the other variables is close to zero or equal to zero.

If the new variables include important features, the decision tree can utilize them to improve accuracy. However, if most of them are noise, KNN is more susceptible to the influence of noise, which can actually lead to a decrease in accuracy.

Part 2

1.

```
trumpbernie <- fread(file = "~/Documents/ML/Labs/Lab2/trumpbernie2.csv")
trumpbernie$trump_tweet <- factor(trumpbernie$trump_tweet, levels = c(0,1))
```

2.

```
set.seed(1984)
tc <- caret::trainControl(method = 'cv', number = 5)
rpart_model_trump <- caret::train(trump_tweet ~ .,
                                  data = trumpbernie,
                                  tuneGrid=expand.grid(cp=c(0,0.001,0.01,0.1,0.25)),
                                  control = rpart.control(minbucket=10),
                                  method = "rpart",
                                  trControl = tc)
```

```
rpart_model_trump
```

CART

1003 samples
780 predictor
2 classes: '0', '1'

No pre-processing
Resampling: Cross-Validated (5 fold)
Summary of sample sizes: 802, 803, 802, 803, 802
Resampling results across tuning parameters:

cp	Accuracy	Kappa
----	----------	-------

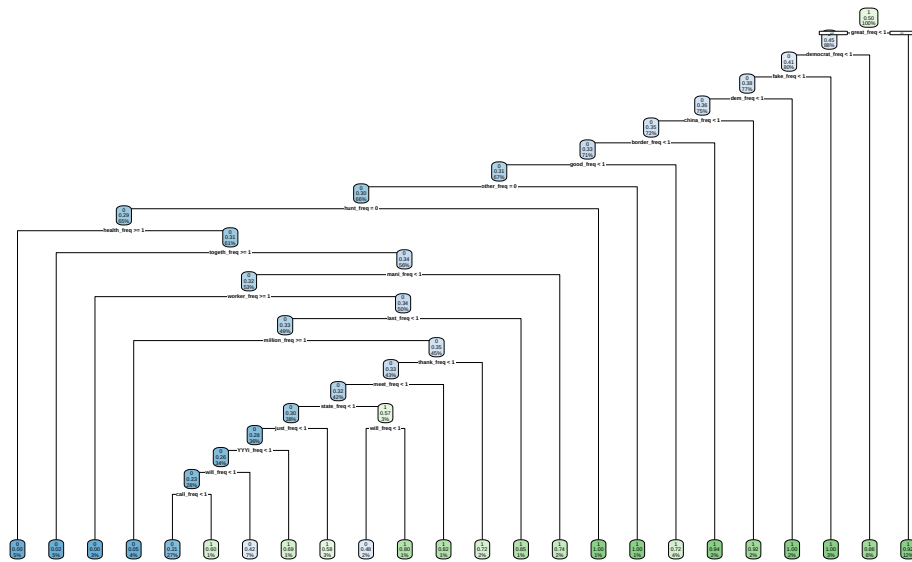
0.000	0.7567214	0.5132777
0.001	0.7607015	0.5212290
0.010	0.7257811	0.4526825
0.100	0.6221393	0.2478545
0.250	0.5034925	0.0000000

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was $cp = 0.001$.

2a

```
rpart.plot(rpart_model_trump$finalModel)
```

Warning: labs do not fit even at cex 0.15, there may be some overplotting



This tree is a big tree.

2b

```
best_decision_tree <- rpart_model_trump$finalModel
varimp_object <- caret::varImp(best_decision_tree)
varimp_dt <- data.table(var=rownames(varimp_object),
                        imp=varimp_object$Overall)
varimp_dt <- varimp_dt[order(imp,decreasing = T)]
varimp_dt[,imp := imp / max(imp)] # Making variable importance into a relative measure
sum(varimp_dt$imp!=0)
```

[1] 43

There are 43 non-zero variables.

The performance is worse than Ridge regression because there are too many noise variables in the dataset, and decision trees are prone to overfitting in the presence of a large number of noise variables.. In contrast, Ridge regression uses regularization to suppress the influence of noise variables on the model.

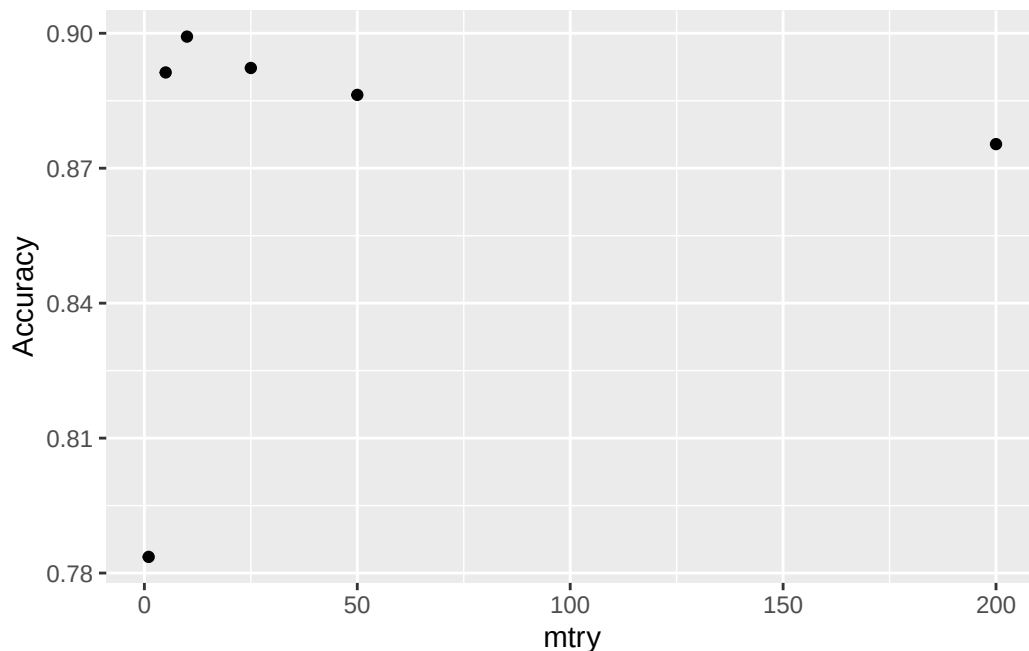
The performance is better than linear regression because decision trees can capture nonlinear relationships and interaction effects, whereas linear regression assumes linearity and cannot capture nonlinear relationships and interaction effects.

3.

3a

```
tc <- caret::trainControl(method = 'cv', number = 5)
mtry_values <- c(1, 5, 10, 25, 50, 200)
set.seed(1984)
rf_model_trump <- caret::train(trump_tweet ~ .,
                              data = trumpbernie,
                              tuneGrid=expand.grid(mtry=mtry_values),
                              method = "rf",
                              ntree = 200,
                              trControl = tc)

ggplot(rf_model_trump$results, aes(x = mtry,y = Accuracy)) + geom_point()
```



As *mtry* (the number of predictors randomly selected at each split) increases, the accuracy first rises and then falls, reaching its maximum at $mtry = 10$.

Considering that the dataset used is high-dimensional, meaning it contains many noisy variables and only a few strong signal variables, using all variables to grow the trees would lead to highly correlated trees. Therefore, decorrelating the trees indeed helps improve performance.

3b

```
best_rf_model <- rf_model_trump$finalModel

oob_error <- best_rf_model$err.rate[best_rf_model$ntree, "OOB"] # OOB error

oob_error
```

```
OOB
0.09770688
```

```
rpart_model_trump$results$Accuracy |> max() ## 0.7607015
```

```
[1] 0.7607015
```

```

varimp_object_rf <- caret::varImp(best_rf_model)
varimp_rf <- data.table(var=rownames(varimp_object_rf),
                        imp=varimp_object_rf$Overall)
varimp_rf <- varimp_rf[order(imp,decreasing = T)]
varimp_rf[,imp := imp / max(imp)] # Making variable importance into a relative measure
sum(varimp_rf$imp!=0)

```

[1] 780

The accuracy of the best random forest model is 0.9022931, while the accuracy of the best decision tree model is 0.7607015, showing a significant performance improvement.

The reason is that decorrelating these trees reduces variance.

Checking variable importance, we can see that 780 variables have non-zero importance.

3c

```

best_rf_model <- rf_model_trump$finalModel

best_rf_model ## 1 - 9.77% = 0.9023

```

Call:

```

randomForest(x = x, y = y, ntree = 200, mtry = param$mtry)
      Type of random forest: classification
      Number of trees: 200

```

No. of variables tried at each split: 10

OOB estimate of error rate: 9.77%

Confusion matrix:

```

      0   1 class.error
0 455  43  0.08634538
1  55 450  0.10891089

```

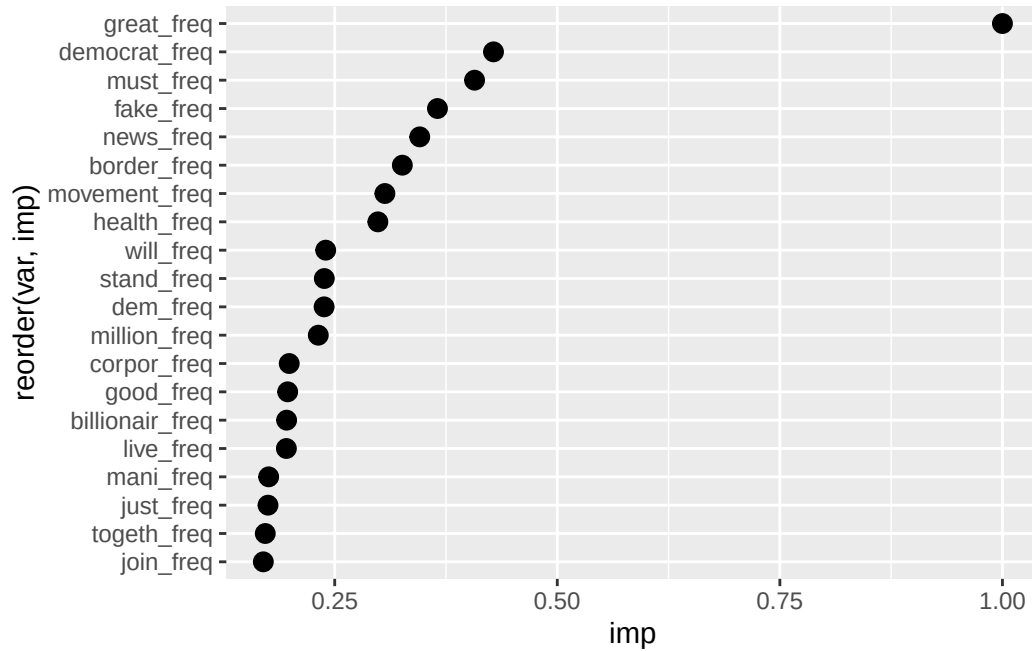
We compare the best random forest model with the ridge regression model from #2, which achieved an accuracy of 86%. As expected, the random forest performs better, achieving an accuracy of 90.23%.

This is because ridge regression cannot reduce the importance of irrelevant variables to exactly zero; it can only shrink them close to zero to reduce variance, but bias still remains. In contrast, random forests can effectively decorrelate the trees, thereby reducing variance.

What does this difference reveal about the data? It suggests that the dataset contains a large number of noisy variables and only a few strong signal variables. It also indicates that the relationship between X and y is very sparse and highly nonlinear.

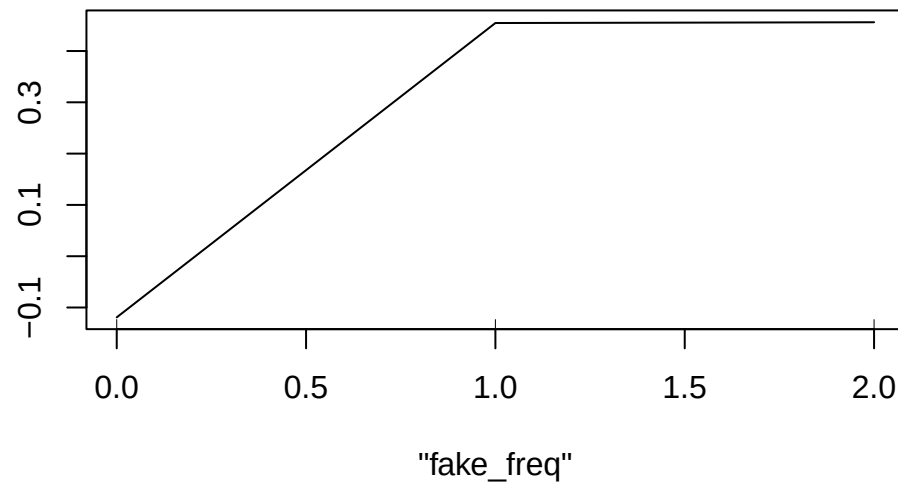
3d

```
ggplot(varimp_rf[1:20], aes(y=reorder(var, imp), x=imp)) + geom_point(size=3)
```



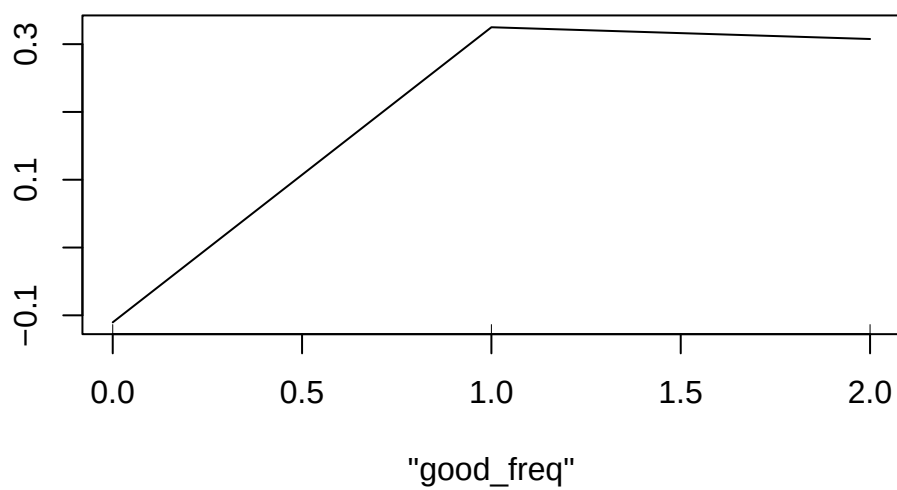
```
randomForest::partialPlot(
  best_rf_model,
  pred.data=trumpbernie,
  x.var='fake_freq',
  which.class = "1")
```

Partial Dependence on "fake_freq"



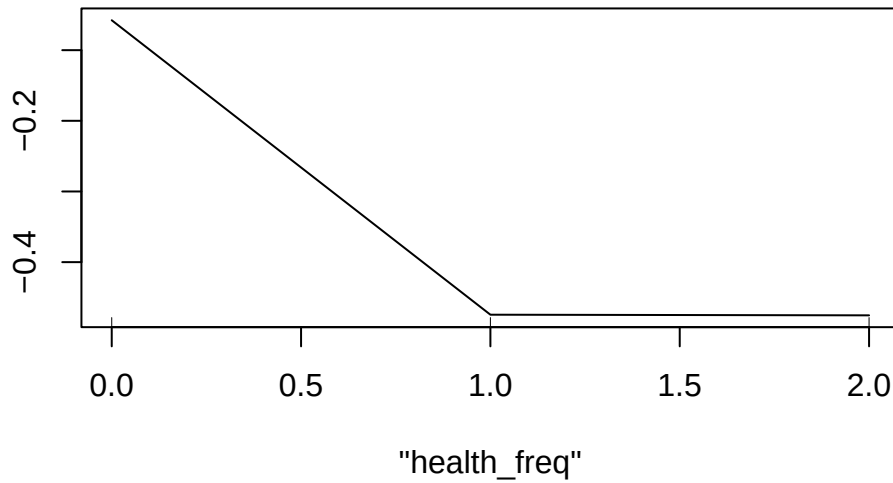
```
randomForest::partialPlot(  
  best_rf_model,  
  pred.data=trumpbernie,  
  x.var='good_freq',  
  which.class = "1")
```

Partial Dependence on "good_freq"



```
randomForest::partialPlot(  
  best_rf_model,  
  pred.data=trumpbernie,  
  x.var='health_freq',  
  which.class = "1")
```

Partial Dependence on "health_freq"



Examining the top twenty variables by importance score, many words characteristic of Trump and Bernie appear, which is largely as expected.

To further investigate whether a word is associated with Trump or Bernie, I selected three words as the variables of interest and plotted their partial dependence plots, specifying `trump_tweet` as the positive class.

The y-axis shows the average deviation in the predicted probability of a Trump tweet when controlling for other variables. A positive y-value indicates that an increase in the word raises the predicted probability of a Trump tweet (Trump style), while a negative y-value indicates that an increase in the word lowers the predicted probability of a Trump tweet, making it more associated with a Bernie tweet (Bernie style).